



IILM UNIVERSITY, GREATER NOIDA

School of Computer Science and Engineering

INTERNSHIP REPORT

ON

**C++ PROGRAMMING AND APPLICATION
DEVELOPMENT**

30 DAYS INTERNSHIP

Submitted To

School of Computer Science and Engineering

IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement for the degree of

B.Tech. in Computer Science and Engineering

Name of Student	DAKSH SHARMA
Class / Section	2CSE23
Roll No.	25SCS1003004699
Year	2nd Year
Semester	3rd Semester
Internship Organisation	Virtual Work Space

Academic Session 2026–27

1. CANDIDATE'S DECLARATION

I, DAKSH SHARMA, hereby declare that the internship report titled “C++ Programming and Application Development – 30 Days Internship” has been completed by me in fulfilment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering. The work documented in this report represents the projects and learning activities completed by me during my internship with Virtual Work Space. I affirm that the report has been prepared for academic submission and that the sources used for technical reference have been appropriately acknowledged.

I further declare that the five C++ applications documented in this report—Basic Calculator, Number Guessing Game, Hotel Management System, Tic-Tac-Toe Game, and Contact Management System—were undertaken as part of my internship work. I understand that I am responsible for ensuring that the submitted material complies with applicable academic and copyright requirements.

Signature: _____

Student Name: DAKSH SHARMA

Roll No.: 25SCS1003004699

2. ACKNOWLEDGEMENT

I express my sincere gratitude to everyone who supported me throughout my 30-day C++ internship with Virtual Work Space. This internship provided me with an opportunity to move beyond classroom concepts and apply C++ programming fundamentals to practical console-based applications.

I am thankful to the internship organisation and the people who provided guidance, constructive feedback, and learning resources during the training period. Their support helped me understand how programming concepts such as functions, loops, conditional statements, arrays, strings, random number generation, file handling, and modular program design can be combined to create useful applications.

I also express my gratitude to the School of Computer Science and Engineering, IILM University, Greater Noida, for encouraging practical learning through internship activities. Finally, I thank my parents for their continuous support and encouragement.

Signature: _____

Student Name: DAKSH SHARMA

Roll No.: 25SCS1003004699

3. INTERNSHIP COMPLETION CERTIFICATE

The internship completion certificate issued by the internship organisation is to be attached on this page as supporting proof of completion.

**[ATTACH ORIGINAL INTERNSHIP COMPLETION CERTIFICATE
HERE]**

Internship Organisation: Virtual Work Space

Internship Topic: C++ Programming

Duration: 30 Days

TABLE OF CONTENTS

Chapter No.	Chapter Name	Page No.
1.	Candidate's Declaration	i
2.	Acknowledgement	ii
3.	Internship Completion Certificate	iii
4.	Project Description	1
4.1	Introduction	1
4.2	Organization Profile	2
4.3	Problem Statement	3
4.4	Project Objectives	4
4.5	Scope of the Project	5
4.6	Technologies and Tools Used	6
4.7	System Architecture	7
4.8	Methodology	9
4.9	Project Implementation and	11

	Description	
4.10	Expected Outcomes and Learning	23
4.11	Certificates and Communication Proof	25
5.	Bibliography / References	26

4. PROJECT DESCRIPTION

4.1 Introduction

The 30-day internship in C++ programming at Virtual Work Space was designed as a practical introduction to software development using C++. The internship focused on converting basic programming concepts into working console-based applications. Instead of limiting the work to isolated coding exercises, the assigned tasks required complete mini-projects with user interaction, input validation, processing logic, output presentation, and, where required, persistent storage.

The five projects completed during the internship were: (1) Basic Calculator, (2) Number Guessing Game, (3) Hotel Management System, (4) Tic-Tac-Toe Game, and (5) Contact Management System. Together, these projects covered fundamental control structures, functions, randomisation, arrays, strings, game logic, file handling, records management, and menu-driven application design.

The work also helped develop a systematic approach to programming: understand the requirement, divide the problem into smaller modules, design the data flow, implement the logic, test normal and incorrect inputs, and refine the program based on observed behaviour.

4.2 Organization Profile

Virtual Work Space was the internship organisation for this 30-day C++ programming internship. The internship was carried out in a virtual learning/work environment and focused on hands-on programming tasks. The assigned activities followed a project-based learning approach in which each task resulted in a functional C++ application.

The internship work was aligned with the development of programming fundamentals expected from a first-year/early-stage Computer Science and Engineering student. The organisation's role in this report is described only in relation to the internship activities completed and the project assignments provided; no additional organisational claims are made beyond the information available for this report.

4.3 Problem Statement

The central problem addressed during the internship was to develop practical programming ability by implementing small but complete C++ applications. A

beginner programmer often understands individual concepts such as if-else statements, loops, arrays, functions, and file streams but may find it difficult to combine them into an application that accepts user input, processes data, handles different cases, and produces reliable output.

The internship therefore used progressively varied tasks. The calculator introduced menu-driven branching and arithmetic operations. The guessing game introduced random number generation and iterative interaction. The hotel management system introduced structured records and file-based persistence. Tic-Tac-Toe focused on arrays and game-state validation. The contact management system extended file handling into a practical record-management workflow.

4.4 Project Objectives

- To strengthen C++ syntax and programming fundamentals through practical implementation.
- To understand menu-driven program design and user interaction.
- To apply conditional statements, loops, functions, arrays, strings, and basic data structures.
- To implement random number generation and repeated user interaction.
- To understand persistent data storage using C++ file handling.
- To develop logical thinking through game-state and winning-condition problems.
- To practise searching, adding, viewing, updating/deleting-style record operations in file-based applications.
- To improve debugging, testing, code organisation, and problem-solving skills.
- To gain experience in converting written requirements into working software.

4.5 Scope of the Project

The scope of the internship was limited to beginner-to-intermediate console applications in C++. The projects were intended for learning and demonstration rather than production deployment. The applications primarily use keyboard input and terminal/console output, with file handling used where persistent storage was part of the requirement.

The Hotel Management System and Contact Management System demonstrate the idea of persistent records through files. In a production system, these applications

could be extended with a database, authentication, concurrency controls, stronger validation, graphical interfaces, and secure data management. Those features are outside the scope of the work documented here.

- User input and console-based interaction.
- Modular menu-driven workflows.
- Basic arithmetic and decision logic.
- Randomised number selection.
- Two-player game interaction and win/draw detection.
- File-based storage and retrieval of records.
- Basic testing and debugging of common input scenarios.

4.6 Technologies and Tools Used

Technology / Tool	Purpose	Application in Internship
C++	Programming language	All five projects
C++ Standard Library	Input/output, strings, utilities and file streams	Across projects
iostream	Console input and output	Menus, prompts and results
fstream	File input/output	Hotel and Contact Management Systems
cstdlib / random utilities	Randomisation support	Number Guessing Game
Arrays / strings	In-memory data representation	Tic-Tac-Toe and record handling
Functions	Modular program design	All projects
Conditional statements	Decision making	All projects
Loops	Repeated operations	All projects
Console / Terminal	User interface	All projects

4.7 System Architecture

The projects follow a simple layered flow suitable for console applications. The user interacts with a menu or prompt, the program validates or interprets the input, a processing module performs the requested operation, and the result is displayed. For the file-based projects, an additional persistence layer reads records from or writes records to a file.

Figure 4.1: General Console Application Flow

USER INPUT → INPUT VALIDATION → APPLICATION LOGIC → RESULT / OUTPUT

For file-based applications: USER INPUT → VALIDATION → RECORD PROCESSING → FILE READ/WRITE → OUTPUT

The architecture is intentionally simple. Each project was designed around the minimum components required to satisfy its functional requirement while keeping the code understandable to a student at the early stage of C++ programming.

4.8 Methodology

A project-based methodology was followed throughout the internship. Each assignment was approached through requirement understanding, planning, implementation, testing, and refinement.

1. Requirement Analysis: Read the task statement carefully and identify the expected inputs, operations, outputs, and special conditions.
2. Design: Break the task into menus, functions, data structures, and processing steps before writing the complete program.
3. Implementation: Write the C++ program using appropriate control structures, functions, arrays/strings, and file streams where required.
4. Testing: Run the application with valid, invalid, boundary, and repeated inputs to observe the behaviour.
5. Debugging: Identify syntax, logical, and runtime issues and modify the code accordingly.
6. Documentation: Record the purpose, working principle, technologies, outcomes, and learning associated with each project.

4.9 Project Implementation and Description

4.9.1 Task 1 – Basic Calculator

The Basic Calculator was the introductory project of the internship. Its purpose was to create a menu-driven C++ program that accepts two numbers, lets the user select an arithmetic operation, and displays the calculated result.

The application demonstrates the use of a menu, switch-case/conditional branching, arithmetic operators, numeric input, and basic error handling for division. A loop can be used to allow the user to perform multiple calculations without restarting the program.

Functional flow: Display menu → Select operation → Enter first number → Enter second number → Perform selected operation → Display result → Continue or exit.

- Addition: $\text{result} = \text{first number} + \text{second number}$.
- Subtraction: $\text{result} = \text{first number} - \text{second number}$.
- Multiplication: $\text{result} = \text{first number} \times \text{second number}$.
- Division: $\text{result} = \text{first number} \div \text{second number}$, with a check to prevent division by zero.
- Exit: terminate the calculator loop.

Learning outcome: This project strengthened understanding of operators, switch-case logic, loops, input/output, and basic validation. It also introduced the importance of considering exceptional cases such as division by zero.

Figure 4.2: Basic Calculator – Functional Flow

MENU → CHOICE → INPUT NUMBERS → OPERATION → RESULT

4.9.2 Task 2 – Number Guessing Game

The Number Guessing Game was developed as an interactive console game in which the computer selects a random number and the user repeatedly attempts to guess it. After every attempt, the program provides a hint indicating whether the guess is too high or too low.

The core logic depends on random number generation, a loop for repeated attempts, comparison operators, and conditional statements. The program terminates when the user guesses the correct number.

- Generate a random target number within a defined range.
- Ask the user to enter a guess.
- Compare the guess with the target.
- Display “Too High” when the guess is greater than the target.
- Display “Too Low” when the guess is smaller than the target.
- Display a success message when the guess matches the target.
- Optionally count the number of attempts to make the game more informative.

Learning outcome: The project improved understanding of randomisation, iterative logic, comparisons, and interactive program flow. It also demonstrated how a simple algorithm can create an engaging user experience.

Figure 4.3: Number Guessing Game – Functional Flow

RANDOM NUMBER → USER GUESS → COMPARE → TOO HIGH / TOO LOW
/ CORRECT → REPEAT

4.9.3 Task 3 – Hotel Management System

The Hotel Management System was a larger application that simulated basic hotel operations. The system was designed to manage room booking information, customer details, and billing information while demonstrating persistent storage using files.

The project required more structured thinking than the earlier programs because several types of information must remain associated with a booking. A typical workflow begins with displaying available operations, accepting a customer or booking request, storing relevant details, retrieving records when needed, and generating billing information.

- Room booking and basic room-status management.
- Customer detail entry and storage.
- Retrieval of stored records.
- Calculation or display of billing information.
- File-based persistence so data can be retained between program executions.
- Menu-driven navigation for hotel operations.

File handling is particularly important in this project. C++ file streams can be used to write records to a file and read them later. This provides a simple persistent-storage mechanism without introducing a database.

Learning outcome: This project strengthened file I/O, record-oriented thinking, menu design, and the relationship between user input, stored data, and application output. It also highlighted the need for validation, such as checking room availability and ensuring that input data is consistent.

Figure 4.4: Hotel Management System – Functional Flow

MENU → BOOK / VIEW / BILL → PROCESS CUSTOMER & ROOM DATA →
FILE STORAGE → DISPLAY RESULT

4.9.4 Task 4 – Tic-Tac-Toe Game

The Tic-Tac-Toe project implemented a two-player console game on a 3×3 grid. Players take turns marking cells, and after each move the program checks whether one player has achieved a winning combination or whether the board has become full.

The project is a practical example of representing a game board in memory and applying logical conditions to detect game states. The 3×3 grid can be represented using a two-dimensional array or another equivalent structure.

- Initialise a 3×3 board.
- Display the board after each move.
- Accept a position from the current player.
- Validate that the selected position is available.
- Place X or O according to the current player.
- Check rows, columns, and diagonals for a winning pattern.
- Check for a draw when all positions are filled without a winner.
- Switch the current player and continue until the game ends.

Learning outcome: The project improved logical reasoning, two-dimensional array usage, state management, nested loops, and condition checking. It also reinforced the importance of validating user choices before changing application state.

Figure 4.5: Tic-Tac-Toe – Functional Flow

BOARD INITIALISATION → PLAYER MOVE → VALIDATE → UPDATE BOARD → CHECK WIN/DRAW → NEXT TURN

4.9.5 Task 5 – Contact Management System

The Contact Management System was developed as a file-based application for adding, viewing, searching, and deleting contact information. The project extended the file-handling concepts used in the Hotel Management System into a focused record-management application.

- Add a new contact and store its information.
- View the contacts stored in the file.
- Search for a contact using an identifying field such as name or phone number.
- Delete a selected contact while preserving the remaining records.

- Use files so that the contact information remains available after the program closes.

A common approach to deletion in a basic file-based program is to read the existing records and write the records that should be retained into a temporary file, then replace the original file. This demonstrates an important practical concept: when direct record deletion is not available, data can be reconstructed while excluding the unwanted record.

Learning outcome: This project developed practical understanding of persistent data, searching, record processing, file streams, and menu-based CRUD-style operations. It also encouraged attention to data consistency and user-friendly prompts.

Figure 4.6: Contact Management System – Functional Flow

MENU → ADD / VIEW / SEARCH / DELETE → FILE PROCESSING →
UPDATED CONTACT DATA

4.9.6 Comparative Summary of the Five Projects

Project	Main Concepts	Input / Output	Key Learning
Basic Calculator	Menu, switch-case, arithmetic, validation	Numbers / result	Control flow and operators
Number Guessing Game	Random number, loops, comparisons	Guess / hints	Interactive iteration
Hotel Management System	Records, menus, file handling	Customer & room data	Persistent data and modular logic
Tic-Tac-Toe	2D array, game state, conditions	Player moves / board	Logical reasoning and state validation
Contact Management System	Files, search, add/view/delete	Contact records	Record management and persistence

4.9.7 Testing and Debugging Approach

Testing was performed by running the programs with representative inputs and checking whether the output matched the expected behaviour. For menu-driven applications, each menu option should be tested individually. For interactive games, multiple turns and terminal conditions should be checked. For file-based systems, records should be verified after writing and again after reopening the application.

- Valid input testing: normal numbers, valid menu choices, valid game positions, and correctly formatted records.
- Boundary testing: extreme or minimum/maximum values where applicable.

- Invalid input testing: unsupported menu options, invalid positions, and inappropriate values.
- Special-condition testing: division by zero, repeated guesses, occupied Tic-Tac-Toe cells, unavailable rooms, and missing search results.
- Persistence testing: close and reopen file-based applications to verify that stored information remains available.
- Regression testing: after fixing an error, rerun previously working features to ensure the change has not introduced a new problem.

4.10 Expected Outcomes and Learning

The internship provided practical exposure to the complete cycle of developing small C++ applications. The expected outcome was not only to produce working programs but also to understand the reasoning behind their structure and behaviour.

- Improved confidence with C++ syntax and standard programming constructs.
- Better understanding of loops, conditional statements, functions, arrays, strings, and operators.
- Ability to design simple menu-driven console applications.
- Practical understanding of random number generation.
- Ability to represent and validate game states.
- Hands-on exposure to file streams and persistent storage.
- Understanding of basic search and record-management workflows.
- Improved debugging and testing habits.
- Experience in breaking a problem statement into manageable programming modules.
- Improved documentation and technical communication skills.

Overall, the five projects provided a progression from a simple arithmetic program to applications involving persistent data and more complex logic. The sequence helped connect foundational C++ concepts with practical software development.

4.10.1 30-Day Internship Work Plan

Period	Activity	Major Learning / Deliverable
Days 1–3	C++ environment, syntax, input/output and problem understanding	Programming setup and fundamentals
Days 4–7	Task 1 – Basic Calculator	Menu-driven arithmetic

		application
Days 8–11	Task 2 – Number Guessing Game	Randomisation and iterative interaction
Days 12–19	Task 3 – Hotel Management System	Records, menus, file storage and billing logic
Days 20–23	Task 4 – Tic-Tac-Toe	2D board, game-state and win detection
Days 24–27	Task 5 – Contact Management System	File-based contact records and operations
Days 28–29	Testing, debugging and refinement	Error handling and verification
Day 30	Documentation and internship report preparation	Final project documentation

4.10.2 Challenges and Solutions

Challenge	Approach / Solution
Designing menu-driven programs	Use clear menu options and a loop that repeatedly accepts a choice until Exit is selected.
Handling incorrect input	Validate choices and important values before processing.
Division by zero	Check the denominator before performing division.
Random guessing logic	Generate a target value once and compare each subsequent guess against it.
Tic-Tac-Toe win detection	Check all rows, columns, and diagonals after every valid move.
Persistent records	Use file streams for reading and writing records and verify data after saving.
Deleting file-based records	Rebuild the stored data while excluding the selected record.

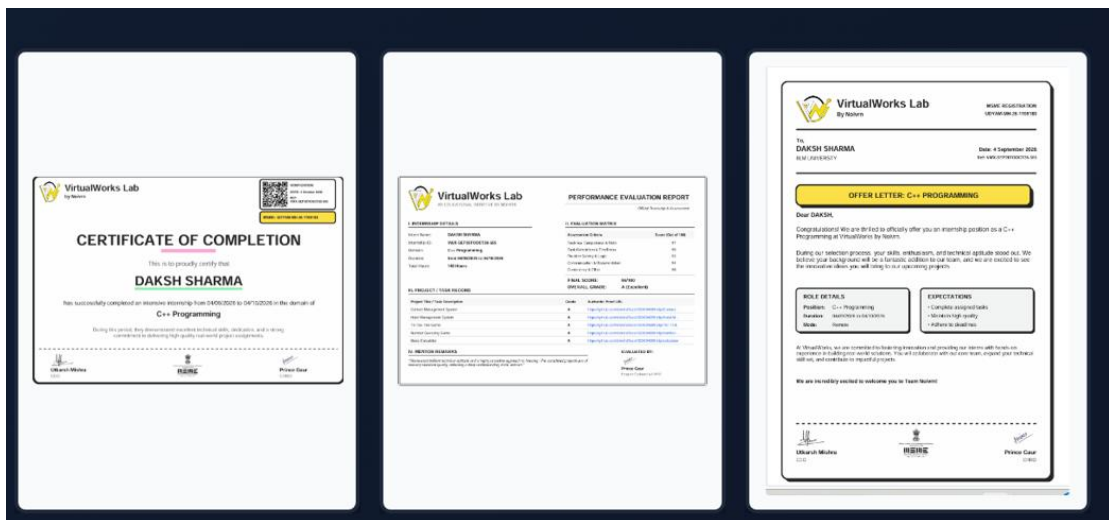
4.10.3 Future Enhancements

Although the internship applications were designed as console-based learning projects, each can be extended into a more complete software product. Possible enhancements include graphical user interfaces, stronger input validation, object-oriented design, database integration, authentication, reporting, and improved exception handling.

- Calculator: scientific functions, calculation history, and GUI.
- Number Guessing Game: difficulty levels, scoring, attempt limits, and leaderboards.
- Hotel Management System: database-backed reservations, customer authentication, room categories, payment records, and reporting.
- Tic-Tac-Toe: single-player mode with an AI opponent, score tracking, and configurable board size.
- Contact Management System: edit/update operation, categories, advanced search, encrypted storage, and database support.

4.11 Certificates and Communication Proof

The internship report format provided by the School of Computer Science and Engineering specifies that certificates of completion and relevant communication mail proof should be attached. The present report therefore reserves this section for the original internship completion certificate and any relevant internship communication evidence



Note: The certificate/communication proof was not included in the source document supplied for preparation of this report. It should be inserted before final submission if available.

5. BIBLIOGRAPHY / REFERENCES

7. C++ language and standard library reference: <https://en.cppreference.com/>
8. IILM University, Greater Noida – official website: <https://iilm.edu/greater-noida/>
9. Tutorial reference supplied with Task 1 – Basic Calculator: <https://www.youtube.com/watch?v=HTv-TgYaXI8>
10. Tutorial reference supplied with Task 2 – Number Guessing Game: <https://www.youtube.com/watch?v=HarwDzStkLY>
11. Tutorial reference supplied with Task 3 – Hotel Management System: <https://www.youtube.com/watch?v=mNvIqarFBts>
12. Tutorial reference supplied with Task 4 – Tic-Tac-Toe Game: <https://www.youtube.com/watch?v=yDfTA1BLuOk>
13. Internship task descriptions supplied for Task 5 – Contact Management System.

END OF REPORT